

Grant Miller

EE 446

10 May 2026

Lab 6 Report

Link to Edge Impulse Project: <https://studio.edgeimpulse.com/studio/992398>

Section 1: Edge Impulse Results

1. The raw inputs used in this Edge Impulse Project are time series data of acceleration in the X, Y, and Z directions/axes.
2. The type of features passed into the NN classifier are spectral features which are extracted from the accelerometer time series data. There are a total of 33 input features including...
 1. accX Peak 1 Height
 2. accX Peak 2 Freq
 3. accZ RMS
 4. accY RMS
 5. accY Peak 3 Height
3. The NN classifier has 3 output features from what I can see on slide 5: nominal, off, and the anomaly score. The classifier itself outputs only 2, nominal and off while the anomaly score comes from the K-means anomaly detection block.
4. The type of features passed into the anomaly detector are spectral features. Only 3 features are used (accX Peak 1 Height, accY Spectral Power 2.0 – 5.0, and accZ Spectral Power 2.0 – 5.0).

5. Screenshot of Edge Impulse model running on the nano and producing inference results:

```
Inferencing settings:
  Interval: 10.00 ms.
  Frame size: 600
  Sample length: 2000 ms.
  No. of classes: 2
Starting inferencing, press 'b' to break
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 609 us, anomaly 512 us, postprocessing 5 ms
#Classification predictions:
  nominal: 0.027344
  off: 0.972656
Anomaly prediction: 8.561289
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 610 us, anomaly 6 ms, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 15.988762
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 609 us, anomaly 6 ms, postprocessing 49 us
#Classification predictions:
```

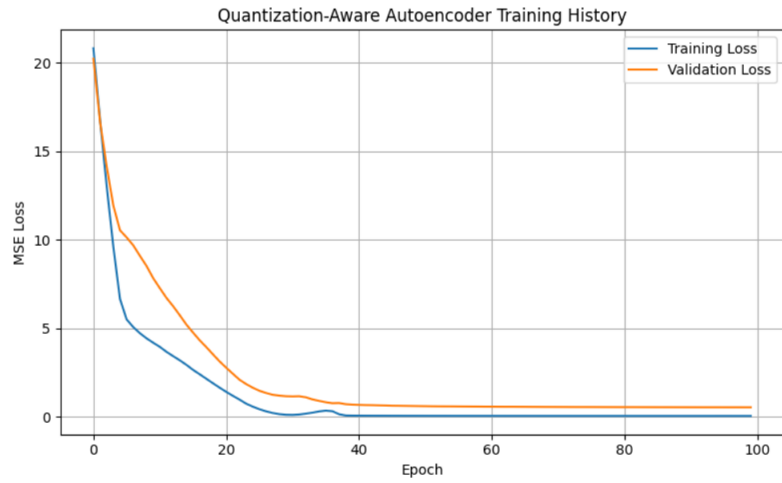
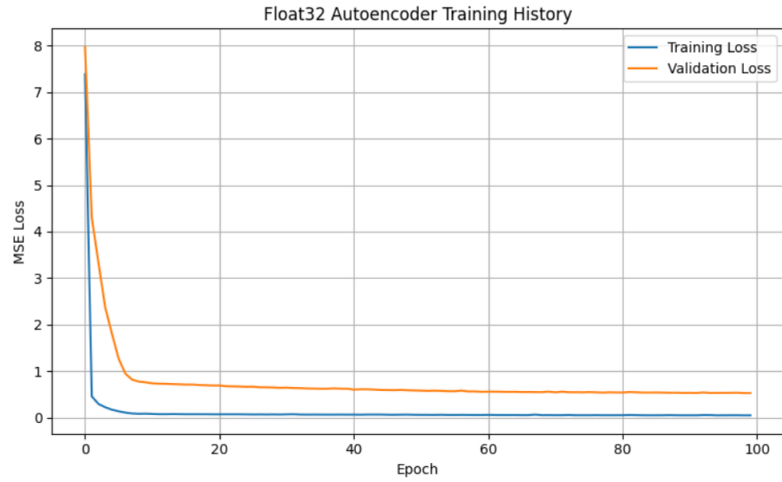
6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

The serial monitor consistently predicted “off” in most samples until I shook the Arduino nano make the prediction spike to an anomaly reading with scores of 151.8 and 1000 indicate the sensor captured motion patterns drastically different from anything in the training set.

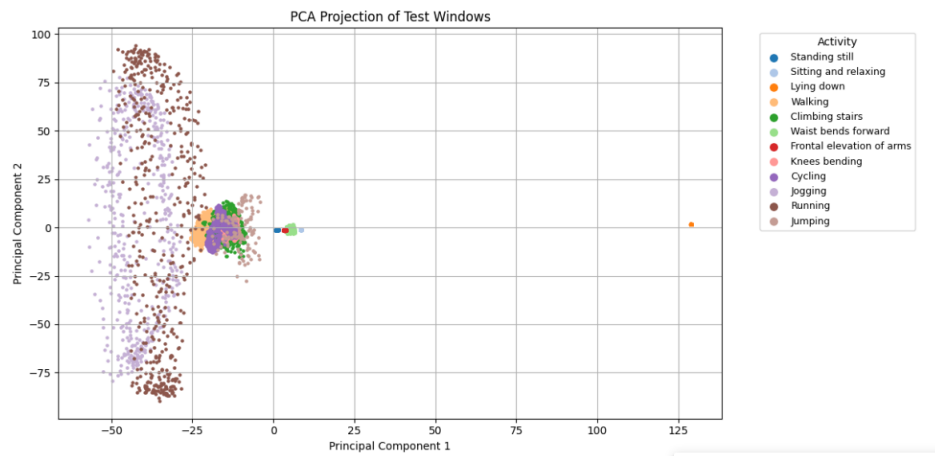
For the most part the classifier output should not be trusted. It was only trained on two classes and is therefore forced to put every input into one of the two classes. If the fan is in a condition not represented by the data, then the classifier will get it wrong almost every time. The classifier output should only be trusted when the anomaly score is low, meaning the input falls near the known clusters. When the anomaly score is high, the classifier's prediction is unreliable.

Section 2: Autoencoder Model Results

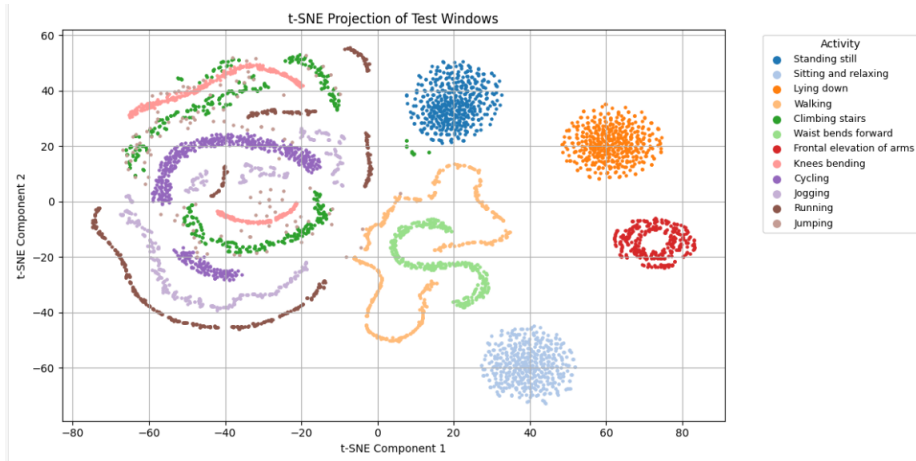
1. Training and validation loss curves



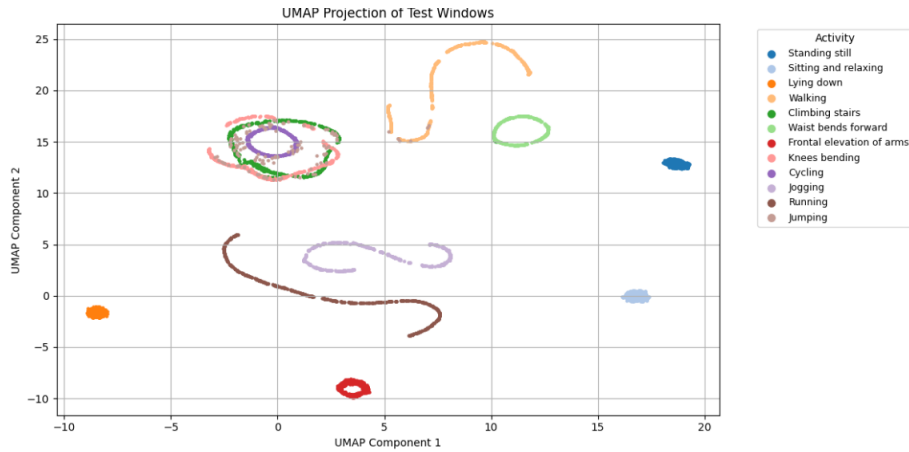
2. PCA visualization



3. t-SNE visualization

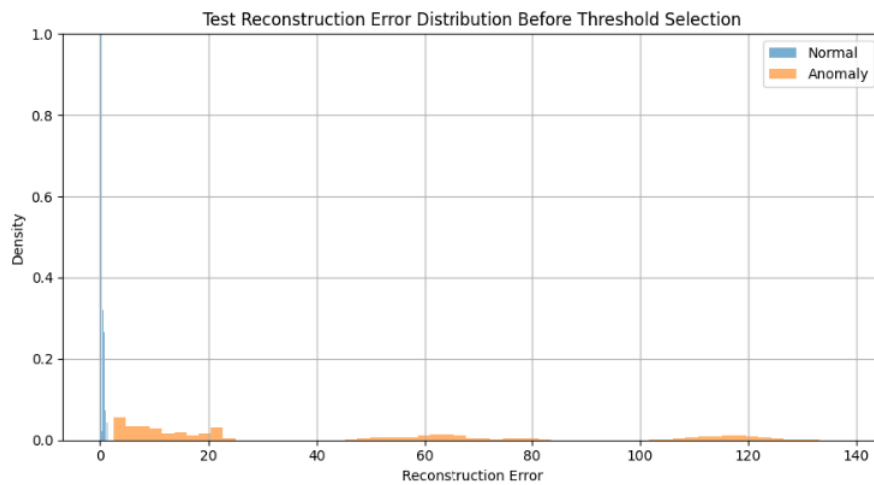


4. UMAP visualization

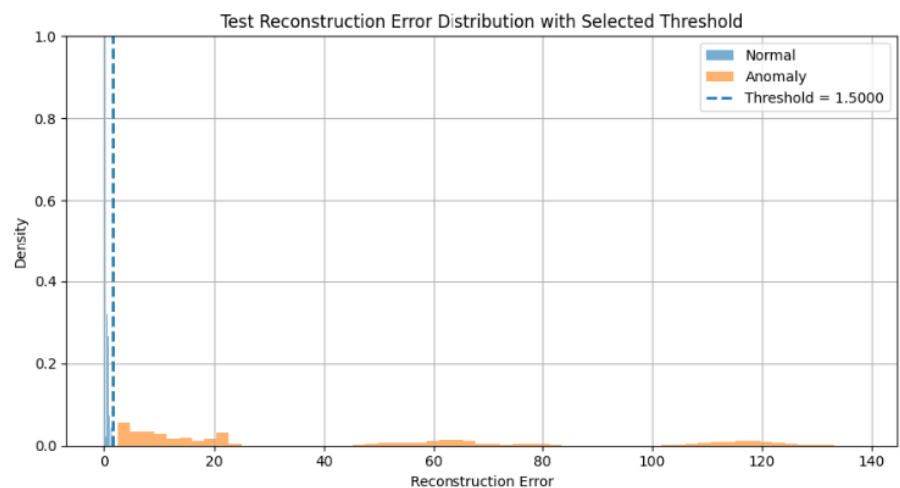


5. Reconstruction error distribution

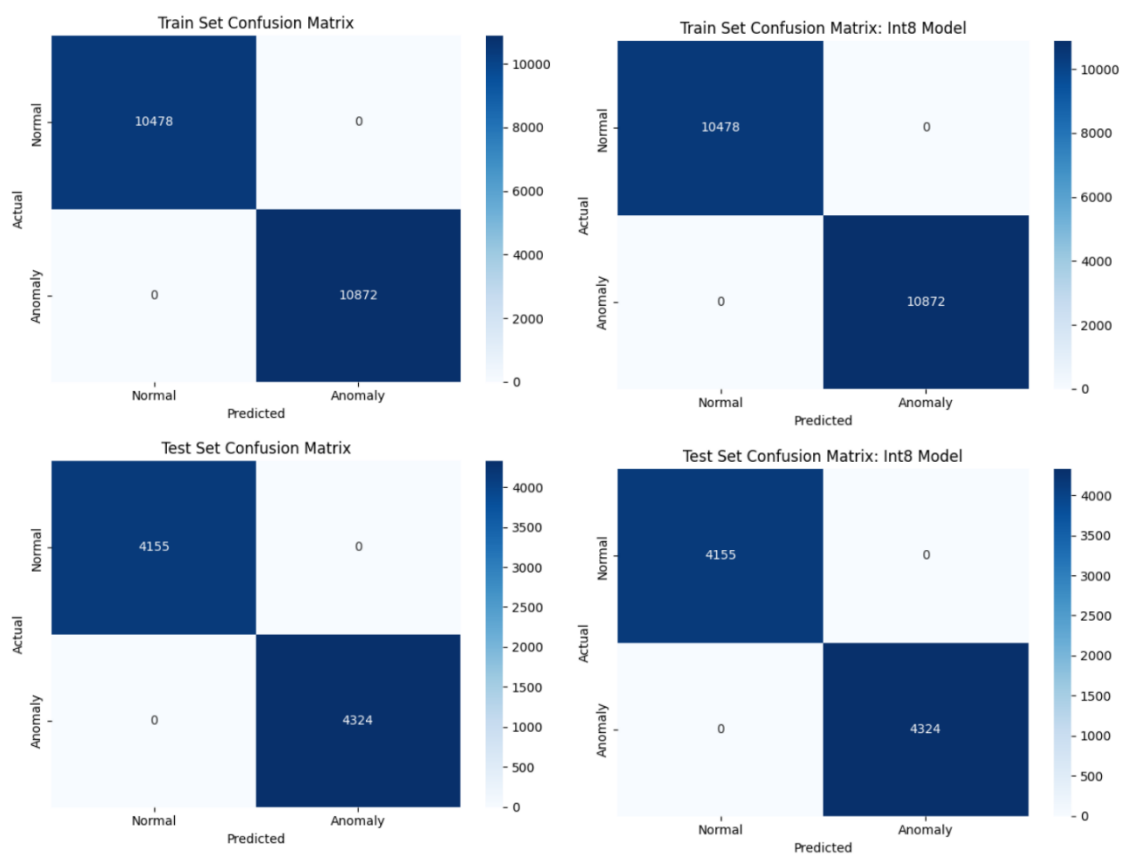
Train normal	mean=0.101933, stderr=0.001731, min=0.007813, max=1.082923
Train anomaly	mean=43.066593, stderr=0.381195, min=2.281693, max=141.087067
Test normal	mean=0.140166, stderr=0.003997, min=0.009268, max=1.364151
Test anomaly	mean=43.432468, stderr=0.637893, min=2.448822, max=137.751648



6. Selected anomaly threshold = 1.5



7. Confusion matrix



8. Classification report or evaluation metrics

Train set evaluation					Train set evaluation: int8 model				
	precision	recall	f1-score	support		precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	10478	Normal (0)	1.00	1.00	1.00	10478
Anomaly (1)	1.00	1.00	1.00	10872	Anomaly (1)	1.00	1.00	1.00	10872
accuracy			1.00	21350	accuracy			1.00	21350
macro avg	1.00	1.00	1.00	21350	macro avg	1.00	1.00	1.00	21350
weighted avg	1.00	1.00	1.00	21350	weighted avg	1.00	1.00	1.00	21350

Test set evaluation					Test set evaluation: int8 model				
	precision	recall	f1-score	support		precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	4155	Normal (0)	1.00	1.00	1.00	4155
Anomaly (1)	1.00	1.00	1.00	4324	Anomaly (1)	1.00	1.00	1.00	4324
accuracy			1.00	8479	accuracy			1.00	8479
macro avg	1.00	1.00	1.00	8479	macro avg	1.00	1.00	1.00	8479
weighted avg	1.00	1.00	1.00	8479	weighted avg	1.00	1.00	1.00	8479

9. Arduino serial monitor output showing reconstruction error and normal/anomaly detection results

```
IMU initialized.
Allocating tensors.
AllocateTensors successful.
Model setup complete.
Reconstruction error: 0.122661
Result: Normal activity
Reconstruction error: 0.122525
Result: Normal activity
Reconstruction error: 0.122524
Result: Normal activity
```

```
Reconstruction error: 1.811840
Result: Anomaly detected
Reconstruction error: 1.699731
Result: Anomaly detected
Reconstruction error: 1.649350
Result: Anomaly detected
Reconstruction error: 1.638973
Result: Anomaly detected
Reconstruction error: 1.480294
Result: Normal activity
Reconstruction error: 1.359596
Result: Normal activity
Reconstruction error: 1.322782
Result: Normal activity
Reconstruction error: 1.195243
Result: Normal activity
```

Section 3: Discussion Questions

1. What threshold did you choose for anomaly detection, and why?

The threshold I chose for anomaly detection was 1.5 because looking at the below data, 1.5 sits right between the max normal and min anomaly making it a good choice.

Train normal mean=0.101933, stderr=0.001731, min=0.007813, max=1.082923

Train anomaly mean=43.066593, stderr=0.381195, min=2.281693, max=141.087067

Test normal mean=0.140166, stderr=0.003997, min=0.009268, max=1.364151

Test anomaly mean=43.432468, stderr=0.637893, min=2.448822, max=137.751648

2. How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder's training phase is done using only normal activity, so it learns how to reconstruct normal activity patterns with very low errors. Now when an anomaly is introduced, it has never seen those patterns before and cannot reconstruct them accurately. So, reconstruction acts as an inherent classifier of normal vs anomalous activity.

3. What information from the Python notebook must be preserved when deploying the model to Arduino?

The quantized model bytes (exported as `autoencoder_model.cc`), the reconstruction error threshold value of 1.5, the input/output quantization parameters which the Arduino sketch uses to convert between float and int8 representations, the window size (100 samples), the input dimensionality ($300 = 100 \times 3$ axes), and the flattening order (samples interleaved as `accX`, `accY`, `accZ` per timestep). These need to match exactly between the notebook and the sketch.

4. Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The model that was trained in the jupyter notebook was trained on data that had a very specific dimensionality (100-sample windows from 3 accelerometer axes, flattened into 300-element vectors in a particular axis ordering). If the Arduino sketch collects data at a different rate, uses a different window size, reorders the axes, etc., the input distribution will differ from what the model learned during training. This would make reconstruction errors higher even when the model isn't seeing an anomaly.

5. What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The model only distinguishes "normal" from "not normal" activity. It cannot identify which specific anomalous activity is occurring. The fixed threshold may not generalize well to new environments or users whose motion patterns differ from the training data. The system also cannot adapt or retrain on-device, so if normal behavior changes over time, the threshold and model become stale without redeployment.